



北京師範大學
BEIJING NORMAL UNIVERSITY

信息安全基础 实验报告

姓名 杨博文 学号 202311081015
院系 人工智能学院 专业 计算机科学与技术

2025 年 4 月 16 日

摘 要

信息安全基础的第二次实验，主要内容为单向哈希函数和 MAC。

目录

1	运行环境及运行方式	3
2	任务 1: 生成消息摘要和 MAC	3
2.1	任务要求	3
2.2	实验结果	3
3	任务 2: 密钥散列消息认证码	3
3.1	任务要求	3
3.2	实验结果	4
4	任务 3: 单向哈希的随机性	4
4.1	任务要求	4
4.2	实验结果	4

1 运行环境及运行方式

硬件为 Macbook Air, M2 芯片, 操作系统为 MacOS Sequoia 15.0。虚拟机软件为 VMWare Fusion, 操作系统为 Linux Ubuntu 20.04。

本实验报告使用 Overleaf 在线生成 L^AT_EX。

2 任务 1: 生成消息摘要和 MAC

2.1 任务要求

在这个任务中, 我们将使用各种单向哈希算法。可以使用以下 openssl dgst 命令为文件生成哈希值。指令: openssl dgst [dgsttype] [filename]

请将 [dgsttype] 替换为特定的单向哈希算法, 如-md5、-sha1、-sha256 等。在这个任务中, 你应该尝试至少 3 种不同的算法, 并描述你的观察结果。

2.2 实验结果

我们使用的明文文件内容为: My name is Bowen Yang. My student No. is 202311081015.

我们使用了-md5、-sha1、-sha256 共 3 种不同的加密方法。

以下是各种方法加密后得到的十六进制密文。

-md5: 3152eec3aed4a36ad24bbe344e6abed1, 长度为 16 字节。

-sha1: bb23261e48cace058337e9612a057fbf02937f83, 长度为 20 字节。

-sha256: 65e3626411f8d082d91a0124a28965759358645b4a38cdc645e4cdd77b0ace5e, 长度为 32 字节。

3 任务 2: 密钥散列消息认证码

3.1 任务要求

在这个任务中, 我们想要为一个文件生成一个密钥散列消息认证码。我们可以使用-hmac 选项 (该选项目前没有文档, 但 openssl 支持它)。下面的示例使用 HMAC-MD5 算法为文件生成一个带有密钥的哈希。-hmac 选项后面的字符串是密钥。

openssl dgst -md5 -hmac "abcdefg" [filename]

请使用 HMAC-MD5、HMAC-SHA256 和 HMAC-SHA1 为你选择的文件生成一个带有密钥的哈希。请尝试几个不同长度的密钥。是否必须在 HMAC 中使用一个固定大小的密钥。如果是, 密钥大小是多少? 如果不是, 为什么?

3.2 实验结果

我们对于 HMAC-MD5 使用不同长度、内容的密钥进行了比较, 比较结果见[表 1](#)。

密钥	HMAC-MD5
0	0be1a7b6b60a85ea77eaf8fce09fdbe0
000	c4b30f6775087afcbdbb272cc4007bb2
012345657	8acf0adee14617214febb054e97e2d41
00112233445566778899aabbccddeeff	0e84b815c7f14b644949304a0b445da9

表 1: HMAC-MD5 使用不同密钥得到的哈希值

在表中我们可以看到, 对于长度不同、内容不同的密钥, 哈希值关系看不出来。

对于同一个密钥, 我们使用了三个不同的加密算法, 见[表 2](#)。

密钥	00112233445566778899aabbccddeeff
加密算法	哈希值
HMAC-MD5	0e84b815c7f14b644949304a0b445da9
HMAC-SHA1	255b90d5be1927a3e46169ac0211745bf65ea238
HMAC-SHA256	26b674745233c178d8c7c30ac060a46697043784a5bd625dc761ab48544f4bae

表 2: 同密钥不同加密算法得到的哈希值

4 任务 3: 单向哈希的随机性

4.1 任务要求

为了理解单向哈希函数的属性, 我们需要对 MD5 和 SHA256 做以下练习:

1. 创建任意长度的文本文件。
2. 使用特定的哈希算法为该文件生成哈希值 H1。
3. 修改文件的一位 (不是一个字节!). 你可以使用 ghex 实现此修改。
5. 为修改后的文件生成哈希值 H2。
6. 请观察 H1 和 H2 是否相似, 并实验报告中描述你的观察结果。你需要编写一个简短的程序来计算 H1 和 H2 之间有多少位相同。

4.2 实验结果

如[图 1](#), 我们将文件的最后一位从 0 改为 1, 即将对应十六进制数从 E 改成 F。进行对应哈希后得到结果如[表 3](#)。

文件	加密算法	哈希值
原	MD5	3152eec3aed4a36ad24bbe344e6abed1
改	MD5	5784338f18347e80fa9bb91cd099609e
原	SHA256	65e3626411f8d082d91a0124a28965759358645b4a38cdc645e4cdd77b0ace5e
改	SHA256	6ff73bfb054531fc734ea1b9d946ae85843afb3ec7f0f08b4c76536f90cbf971

表 3: 任务 3 结果

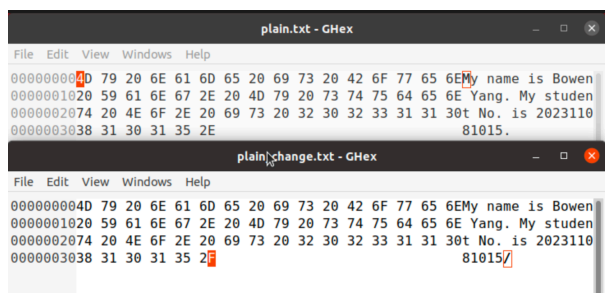


图 1: 任务 3: 更改最后一位

我们写了一段 python 代码，对结果进行比较。输出如下：

MD5 哈希比特相同位数: 59/128，相同率: 46.09%

SHA256 哈希比特相同位数: 123/256，相同率: 48.05%

我们可以发现，虽然原文只改了 1 位，但加密后的哈希值天差地别，基本可以认为二者没有关系（因为两个随机串比特相同率期望为 50%）。这是因为哈希的输入只要稍有变化，输出几乎完全不同这一特性。